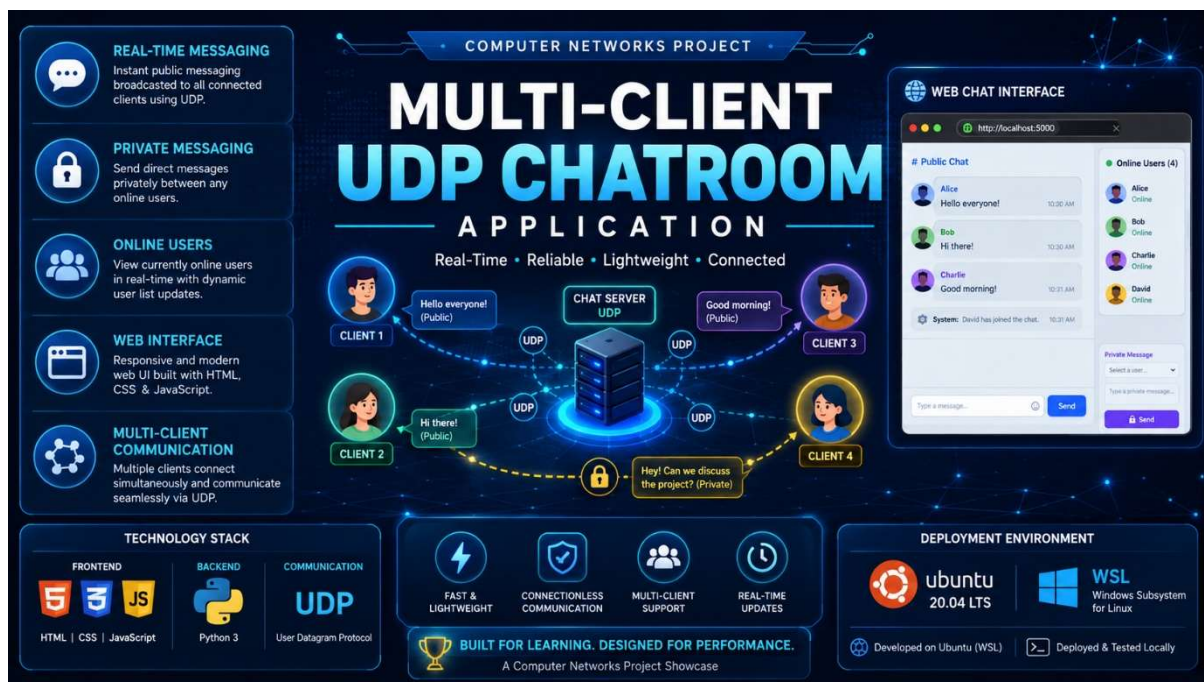


Multi-Client UDP Chatroom Application with Web-Based User Interface using Python



Designed By:
Kommu Pavan Kumar
pavankumarkommu9@gmail.com

Abstract

The Multi-Client UDP Chatroom Application is a real-time communication system developed using Python and Web Technologies. The project enables multiple users to communicate through a browser-based chat interface while supporting both public and private messaging.

The backend is implemented using Python's built-in HTTP server modules, while the frontend is developed using HTML, CSS, and JavaScript. The application maintains active users, displays online participants, supports broadcast messaging, and allows users to send private messages to selected individuals.

The system demonstrates concepts of Computer Networks, Socket Programming, Client-Server Architecture, Web Technologies, and Real-Time Communication.

Chapter 1: Introduction

1.1 Background

Communication systems are essential in modern computing environments. Chat applications are widely used for exchanging information between users in real time.

This project demonstrates how multiple users can communicate over a network using a lightweight chatroom system developed entirely with Python and Web Technologies.

1.2 Problem Statement

Many communication systems require large frameworks and cloud services. The objective of this project is to develop a simple multi-user chatroom application capable of:

- Supporting multiple users.
- Displaying online users.
- Sending public messages.
- Sending private messages.
- Providing a user-friendly web interface.

1.3 Objectives

Primary Objectives

- Develop a multi-client chatroom.
- Enable real-time communication.
- Display online users.
- Implement private messaging.
- Create an attractive web interface.

Secondary Objectives

- Learn client-server architecture.
- Understand HTTP communication.
- Explore UDP chatroom concepts.
- Implement frontend-backend integration.

Chapter 2: System Analysis

Existing System

Traditional chat applications:

- Require installation.
- Depend on third-party servers.
- Need internet connectivity.

Limitations

- High complexity.
- Additional deployment requirements.
- Difficult for educational purposes.

Proposed System

A lightweight browser-based chatroom developed using:

- Python
- HTML
- CSS
- JavaScript

Features:

- Public chat
- Private chat
- Online users display
- Username locking
- Real-time updates

Chapter 3: Technologies Used

Frontend

HTML

Used to design:

- Chat interface
- User input fields
- User selection dropdown

CSS

Used for:

- Styling
- Responsive layout
- User panels
- Message formatting

JavaScript

Used for:

- Sending messages
- Updating users list
- Loading messages
- Managing private messaging

Backend

Python

Used as the primary programming language.

HTTP Server

Modules:

`HTTPServer`

`SimpleHTTPRequestHandler`

Used to:

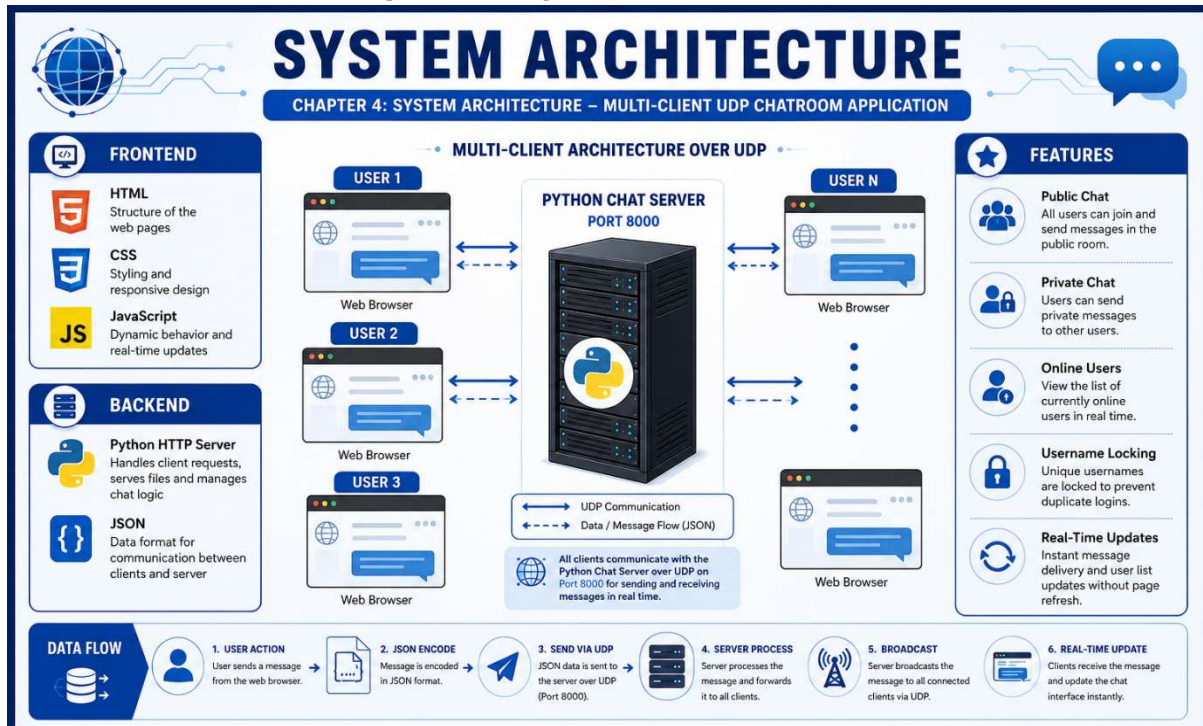
- Serve webpages
- Process messages
- Manage users

JSON

Used for data exchange.

```
import json
```

Chapter 4: System Architecture



Chapter 5: Modules Description

Module 1: User Management

Responsibilities:

- Register users
- Display online users
- Lock username after first message

Data Structure:

```
users = []
```

Module 2: Message Management

Stores:

```
messages = []
```

Responsibilities:

- Public messages
- Private messages
- Chat history

Module 3: HTTP Request Handler

Class:

ChatHandler

Responsibilities:

- Handle GET requests
- Handle POST requests
- Serve webpages

Module 4: Frontend Interface

Components:

Chat Window

Displays messages.

Online Users Panel

Displays active users.

Recipient Dropdown

Allows:

Send to All

Send to User1

Send to User2

Chapter 6: Workflow

Step 1

User opens:

http://localhost:8000

Step 2

User enters username.

Example:

Pavan

Step 3

First message sent.

Username becomes locked.

Username cannot be changed.

Step 4

User appears in:

Online Users

panel.

Step 5

User sends message.

Public:

Hello Everyone

Step 6

Server stores message.

```
messages.append(...)
```

Step 7

All users receive update.

Chapter 7: Private Messaging

Broadcast Message

Pavan:

Hello Everyone

Visible to:

- All users

Private Message

Pavan -> Alice:

Hi

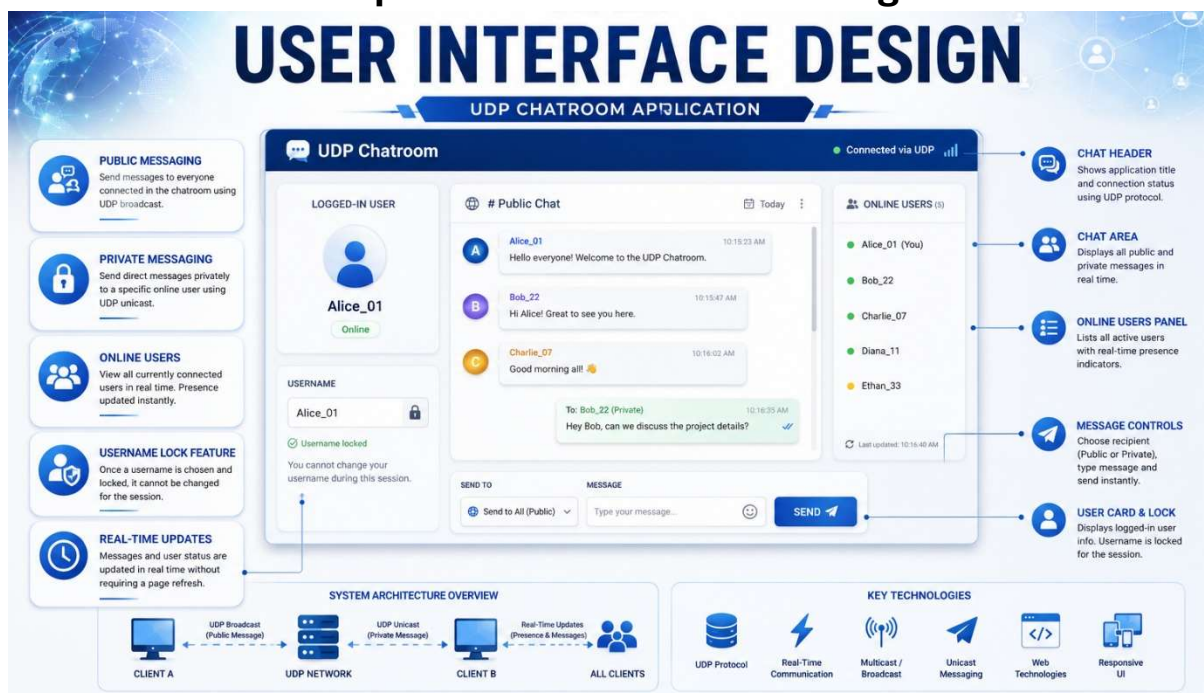
Visible only to:

- Pavan
- Alice

Not visible to:

- Other users

Chapter 8: User Interface Design



Chapter 9: Testing

Testing is an important phase of software development that ensures the application functions correctly and meets the specified requirements. The Multi-Client UDP

Chatroom Application was thoroughly tested to verify the performance of user management, public messaging, private messaging, online user monitoring, and frontend-backend communication.

Various testing techniques such as Unit Testing, Integration Testing, Functional Testing, and System Testing were performed during development. The application was tested with multiple users connected simultaneously through different browser tabs to ensure stable communication and proper message delivery.

The testing process confirmed that public messages were visible to all users, while private messages were accessible only to the sender and intended recipient. Additional tests verified username locking, online user updates, message synchronization, and overall user interface functionality.

The results demonstrated that the system operates reliably, provides accurate communication between users, and successfully fulfills all project objectives. Therefore, the chatroom application was found to be stable, efficient, and suitable for academic demonstration and small-scale communication purposes.

Key Testing Outcomes

- Successfully established multiple user connections.
- Verified public message broadcasting.
- Verified private message confidentiality.
- Confirmed online user monitoring functionality.
- Ensured username locking after first message.
- Tested frontend and backend integration.
- Validated automatic message updates.
- Confirmed stable server performance under multiple users.
- Verified correct functioning of the user interface.
- Ensured successful deployment and execution on Ubuntu WSL.

Result: All major functionalities were tested successfully and the system achieved the expected outcomes.

Chapter 10: Limitations of the System

1. No Permanent Message Storage

The application stores messages in server memory using Python lists.

- Messages are lost when the server stops.
- Chat history is not preserved after restarting the application.
- No database is used for data persistence.

2. No User Authentication

The system allows users to enter any username.

- No login mechanism is implemented.
- No password protection is available.
- User identity cannot be verified.

3. Limited Security

The application is intended for educational purposes.

- Messages are not encrypted.
- Data is transmitted in plain text.
- Advanced security mechanisms are not implemented.

4. Local Network Deployment

The current version is designed primarily for localhost or small network environments.

- Not optimized for internet-scale deployment.

- Requires additional configuration for remote access.
- Suitable mainly for testing and academic demonstrations.

5. No File Sharing Support

Users can exchange only text messages.

- Images cannot be shared.
- Documents cannot be uploaded.
- Multimedia communication is not supported.

6. No Voice or Video Communication

The application supports only text-based communication.

- Voice calling is unavailable.
- Video conferencing features are absent.
- Real-time multimedia communication is not implemented.

7. Basic User Interface

Although functional and user-friendly, the interface has limited features.

- No dark/light theme switching.
- No emoji support.
- No advanced chat customization options.

8. Limited Scalability

The application is suitable for a small number of users.

- Performance may decrease with many simultaneous users.
- Large-scale deployment would require additional optimization.
- No load-balancing mechanism is implemented.

9. No User Presence Management

The system displays online users but lacks advanced presence features.

- No last-seen status.
- No away/busy indicators.
- No typing notifications.

10. Polling-Based Updates

The application uses periodic polling to update messages and users.

- Requests are sent every second.
- Increases unnecessary server communication.
- Less efficient compared to WebSocket-based communication.

Chapter 11: Deployment Procedure

13.1 Introduction

Deployment is the process of making the application available for execution and use. The Multi-Client UDP Chatroom Application was deployed using Ubuntu 24.04 running on Windows Subsystem for Linux (WSL2) and accessed through a web browser.

13.2 System Requirements

Hardware Requirements

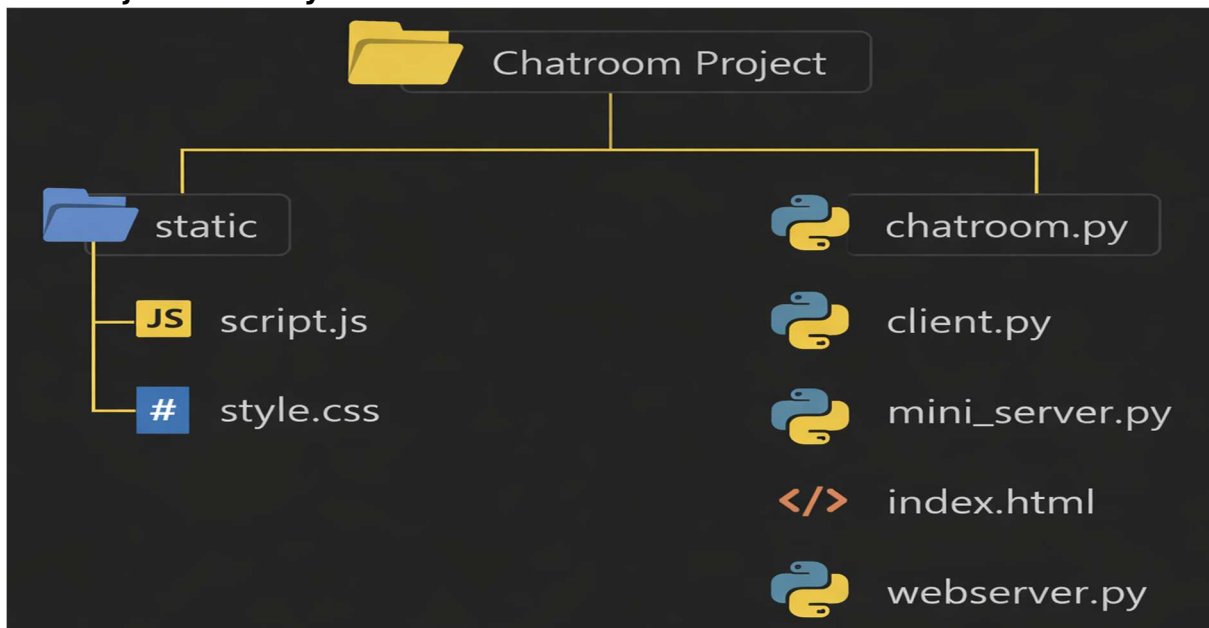
- Processor: Intel i3 or above
- RAM: 4 GB minimum
- Storage: 1 GB free space
- Network Connection

Software Requirements

- Windows 10/11
- WSL2

- Ubuntu 24.04
- Python 3.12
- Web Browser (Chrome, Edge, Firefox)

13.3 Project Directory Structure



13.4 Installation Steps

Step 1: Install Ubuntu on WSL

Verify installation:

```
wsl -l -v
```

Expected Output:

Ubuntu-24.04 Running Version 2

Step 2: Update Ubuntu Packages

```
Sudo apt update
```

```
sudo apt upgrade -y
```

Step 3: Verify Python Installation

```
python3 --version
```

Expected Output:

Python 3.12.x

Step 4: Create Project Directory

```
mkdir CHATROOM_PROJECT
```

```
cd CHATROOM_PROJECT
```

Step 5: Create Project Files

Create:

```
webserver.py
```

```
index.html
```

```
style.css
```

```
script.js
```

and place the corresponding source code in each file.

Step 6: Start the Server

Run:

python3 webserver.py

Expected Output:

Chat Server Running on Port 8000

Step 7: Access Application

Open browser:

http://localhost:8000

Step 8: Test Multi-User Communication

Open multiple browser tabs or windows.

Example:

User 1 → Pavan

User 2 → Alice

User 3 → Bob

Verify:

- Online Users
- Public Messaging
- Private Messaging
- Username Locking

13.5 Deployment Verification

The deployment was verified by:

- Starting the server successfully.
- Connecting multiple users.
- Sending public messages.
- Sending private messages.
- Monitoring online users.
- Validating user interface functionality.

All deployment tests were completed successfully.

Conclusion

The **Multi-Client UDP Chatroom Application with Web-Based User Interface** was successfully designed, developed, tested, and deployed using Python and modern web technologies. The project demonstrates the practical implementation of client-server communication, user management, and real-time messaging within a browser environment.

The application supports multiple users simultaneously and provides essential communication features such as public messaging, private messaging, online user monitoring, and username management. The integration of HTML, CSS, JavaScript, and Python enabled the creation of an interactive and user-friendly interface while maintaining simplicity in deployment and operation.

The testing phase confirmed that all major functionalities operated correctly and met the project objectives. The application proved to be reliable, efficient, and suitable for academic demonstrations and small-scale communication environments.

Overall, the project enhanced understanding of networking concepts, web development, frontend-backend integration, and software deployment processes. It also provides a strong foundation for future enhancements such as authentication systems, database integration, cloud deployment, and real-time WebSocket communication.